

Algorithms

Lecture #15

Syed Hamed Raza

In-Place, Stable Sorting

In-place, Stable Sorting

- An *in-place* sorting algorithm is one that uses no additional array for storage.
- A sorting algorithm is *stable* if duplicate elements remain in the same relative position after sorting.

9	3	3'	5	6	5'	2	1	3''	unsorted
1	2	3	3'	3''	5	5'	6	9	stable sort
1	2	3'	3	3''	5'	5	6	9	unstable

In-Place, Stable Sorting

In-place, Stable Sorting

- Bubble sort, insertion sort and selection sort are in-place sorting algorithms.
- Bubble sort and insertion sort can be implemented as stable algorithms but selection sort cannot (without significant modifications).
- Mergesort is a stable algorithm but not an in-place algorithm. It requires extra array storage.

In-Place, Stable Sorting

In-place, Stable Sorting

- The best we have seen so far is $O(n \log n)$ algorithms for sorting.
- Is it possible to do better than $O(n \log n)$?
- If a sorting algorithm is solely based on comparison of keys in the array then it is *impossible* to sort more efficiently than $\Omega(n \log n)$ time.

Lower Bounds for sorting

Lower Bounds for Sorting

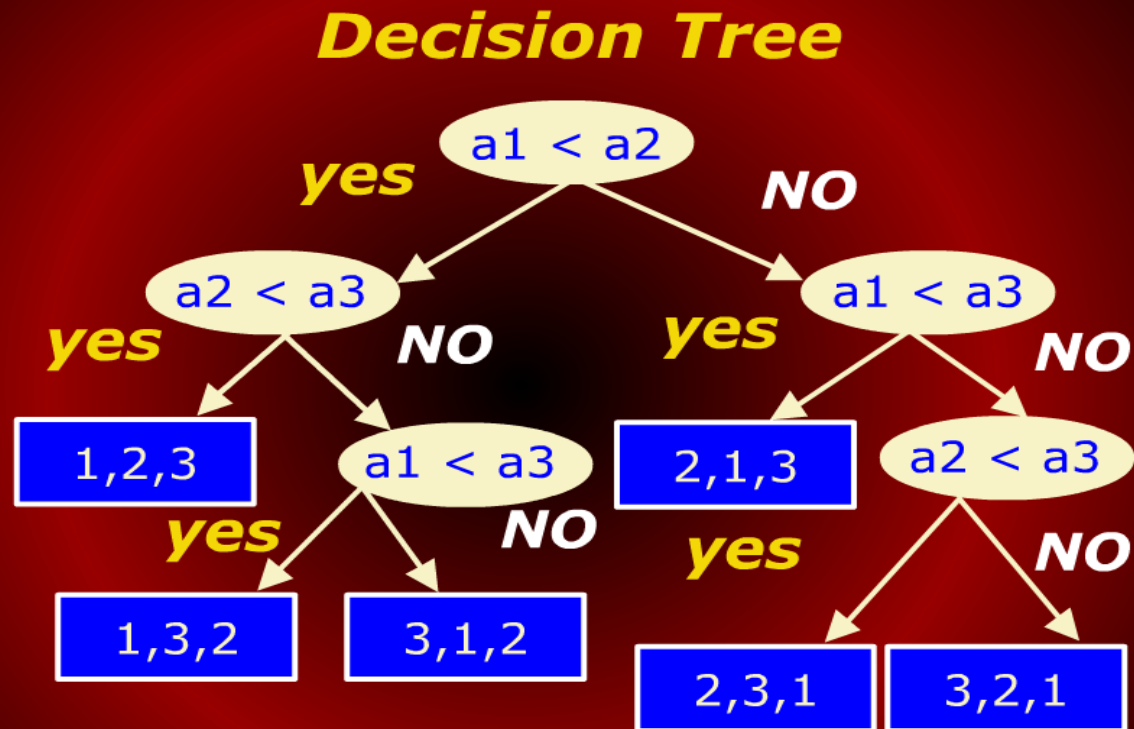
- Consider sorting three numbers a_1 , a_2 , a_3 .
- There are $3! = 6$ possible combinations:
 (a_1, a_2, a_3) , (a_1, a_3, a_2) , (a_3, a_2, a_1)
 (a_3, a_1, a_2) , (a_2, a_1, a_3) , (a_2, a_3, a_1)
- One of these permutations leads to the numbers in sorted order.

Lower Bounds for sorting

Lower Bounds for Sorting

- The comparison based algorithm defines a *decision tree*.
- Here is the tree for the three numbers.

Decision Tree



Lower Bounds for sorting

Lower Bounds for Sorting

- For n elements, there will be $n!$ possible permutations.
- The height of the tree is exactly equal to $T(n)$, the running time of the algorithm.
- The height is $T(n)$ because any path from the root to a leaf corresponds to a sequence of comparisons made by the algorithm.

Lower Bounds for sorting

Lower Bounds for Sorting

- Any binary tree of height $T(n)$ has at most $2^{T(n)}$ leaves.
- Thus a comparison based sorting algorithm can distinguish between at most $2^{T(n)}$ different final outcomes.

- So we have

$$2^{T(n)} \geq n!$$

and therefore

$$T(n) \geq \log(n!)$$

Lower Bounds for sorting

Lower Bounds for Sorting

- We can use Stirling's approximation for $n!$:

$$n! \geq \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

- Therefore

$$\begin{aligned} T(n) &\geq \left(\log \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \right) \\ &= \left(\log \sqrt{2\pi n} \right) + n \log n - n \log e \\ &\in \Omega(n \log n) \end{aligned}$$

Counting Sort

Counting Sort

COUNTING-SORT

(array A, array B,int k)

```
1  for i ← 1 to k
2  do C[i] ← 0 k times
3  for j ← 1 to length[A]
4  do C[A[j]] ← C[A[j]] + 1 n times
5  // C[i] now contains the number
   of elements = i
6  for i ← 2 to k
```

Lower Bounds for sorting

Lower Bounds for Sorting

We thus have the following theorem.

Theorem 1

*Any comparison-based sorting algorithm
has worst-case running time $\Omega(n \log n)$.*

Linear time Sorting

Linear Time Sorting

- The lower bound implies that if we hope to sort numbers faster than $O(n \log n)$, we cannot do it by making comparisons alone.
- Is it possible to sort without making comparisons?
- The answer is yes, but only under very restrictive circumstances.

Counting Sort

Counting Sort

- We will consider three algorithms that are faster and work by not making comparisons.
- Counting sort assumes that the numbers to be sorted are in the range 1 to k where k is small.
- The basic idea is to determine the rank of each number in final sorted array.

Counting Sort

Counting Sort

- Recall that the rank of an item is the number of elements that are less than or equal to it.
- Once we know the ranks, we simply copy numbers to their final position in an output array.
- The algorithm sorts in $\Theta(n + k)$.
- If k is $\Theta(n)$ then counting sort is an $\Theta(n)$ time algorithm.

Counting Sort

The algorithm uses three arrays.

1. $A[1..n]$: Holds the initial input.
2. $B[1..n]$: Array that holds the sorted output.
3. $C[1..k]$: Array of integers. $C[x]$ is the rank of x in A , where $x \in [1..k]$.

Counting Sort

	1	2	3	4	5	6	7	8	9	10	11
Input $A[1..n]$	7	1	3	1	2	4	5	7	2	4	3

$k = 7$

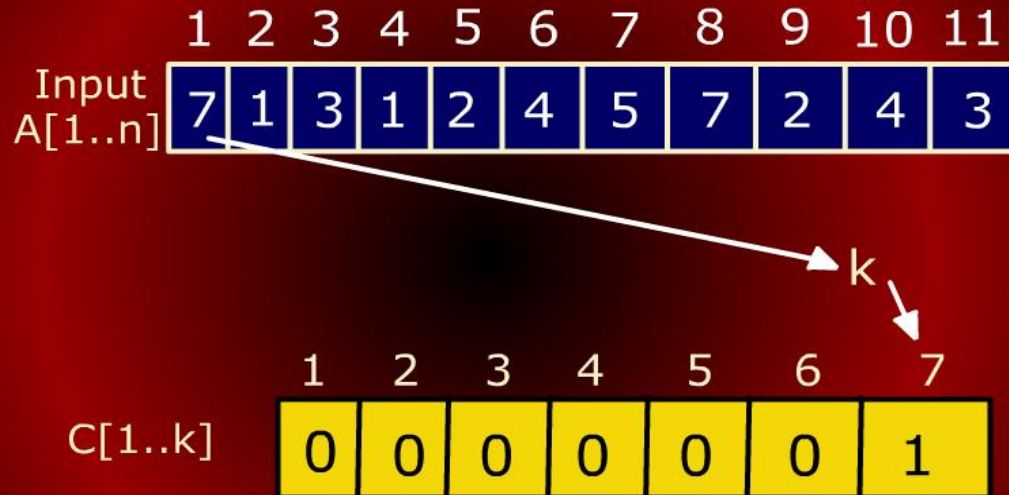
	1	2	3	4	5	6	7
$C[1..k]$	0	0	0	0	0	0	0

Counting Sort

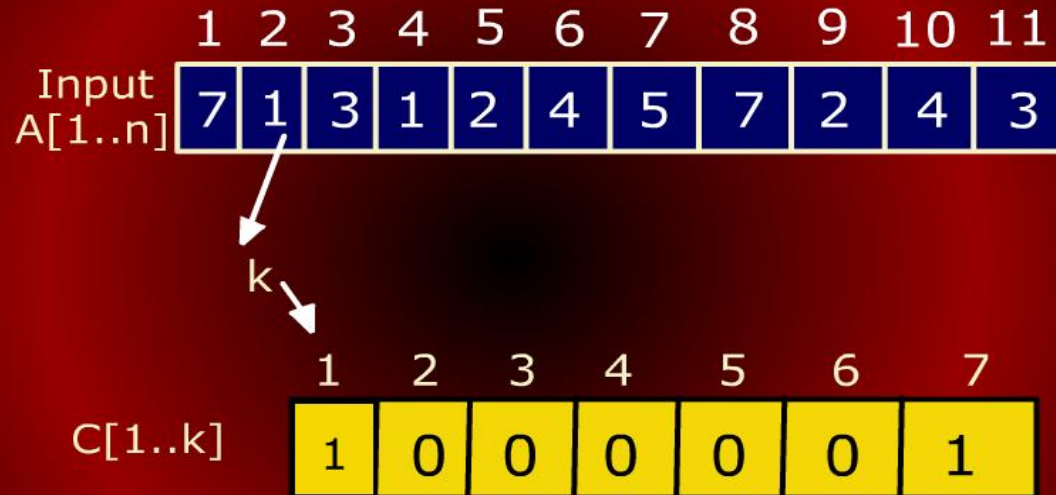
	1	2	3	4	5	6	7	8	9	10	11
Input $A[1..n]$	7	1	3	1	2	4	5	7	2	4	3

	1	2	3	4	5	6	7
$C[1..k]$	0	0	0	0	0	0	0

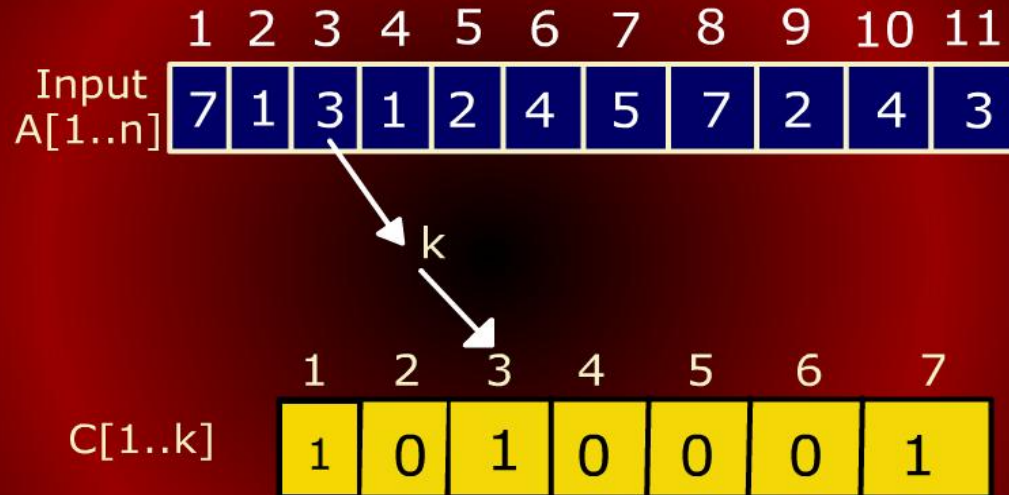
Counting Sort



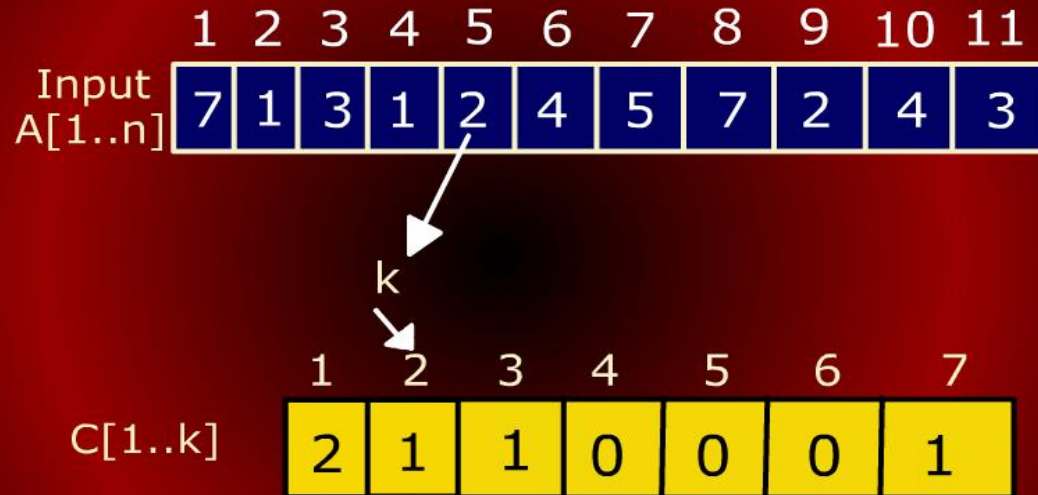
Counting Sort



Counting Sort



Counting Sort



Counting Sort

	1	2	3	4	5	6	7	8	9	10	11
Input A[1..n]	7	1	3	1	2	4	5	7	2	4	3

	1	2	3	4	5	6	7	8	9	10	11
Output B[1..n]						3					

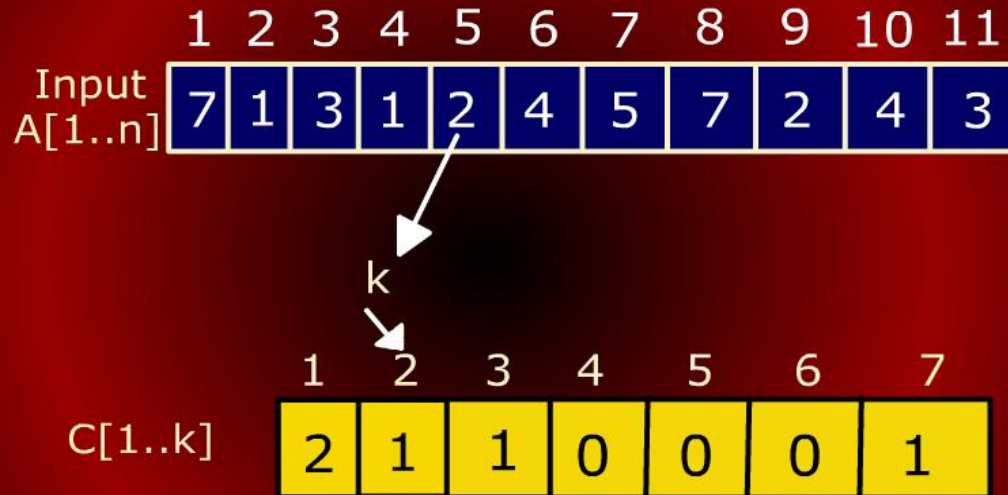
$$B[6] = B[C[3]] = B[C[A[11]]] = A[11] = 3$$

	1	2	3	4	5	6	7
C	2	4	6	8	9	9	11

$$C[A[11]] = C[A[11]] - 1$$

C	2	4	5	8	9	9	11
---	---	---	---	---	---	---	----

Counting Sort



Counting Sort

	1	2	3	4	5	6	7	8	9	10	11
Input A[1..n]	7	1	3	1	2	4	5	7	2	4	3

	1	2	3	4	5	6	7
C[1..k]	2	2	2	2	1	0	2

for i= 2 to 7

do C[i] = C[i] + C[i- 1]

	1	2	3	4	5	6	7
C	2	4	6	8	9	9	11

6 elements ≤ 3

Counting Sort

	1	2	3	4	5	6	7	8	9	10	11
Input A[1..n]	7	1	3	1	2	4	5	7	2	4	3

	1	2	3	4	5	6	7	8	9	10	11
Output B[1..n]						3					

$$B[6] = B[C[3]] = B[C[A[11]]] = A[11] = 3$$

	1	2	3	4	5	6	7
C	2	4	6	8	9	9	11

$$C[A[11]] = C[A[11]] - 1$$

C	2	4	5	8	9	9	11
---	---	---	---	---	---	---	----

Input
A[1..n]

7	1	3	1	2	4	5	7	2	4	3
---	---	---	---	---	---	---	---	---	---	---

Output
B[1..n]

$B[8] = B[C[4]] = B[C[A[10]]] = A[10] = 4$

	1	2	3	4	5	6	7
C	2	4	5	8	9	9	11

$C[A[10]] = C[A[10]] - 1$
 C: 2 4 5 7 9 9 11

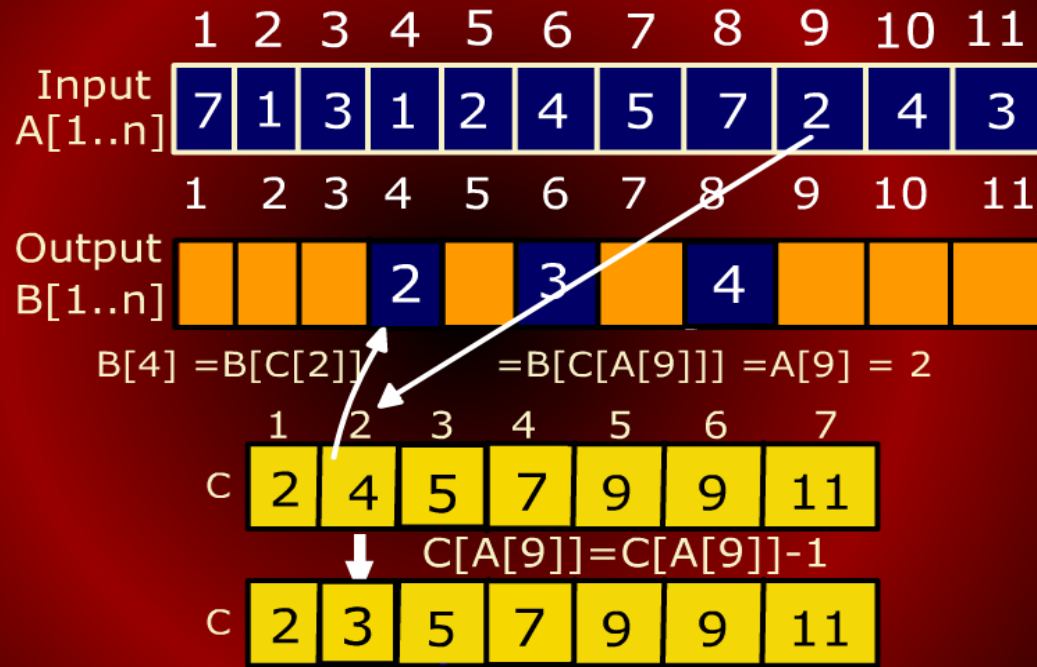
Counting Sort

	1	2	3	4	5	6	7	8	9	10	11
Input $A[1..n]$	7	1	3	1	2	4	5	7	2	4	3

Finally

	1	2	3	4	5	6	7
$C[1..k]$	2	2	2	2	1	0	2

Counting Sort



Counting Sort

	1	2	3	4	5	6	7	8	9	10	11
Input A[1..n]	7	1	3	1	2	4	5	7	2	4	3

	1	2	3	4	5	6	7	8	9	10	11
Output B[1..n]				2		3		4			7

$$B[11] = B[C[7]] = B[C[A[8]]] = A[8] = 7$$

	1	2	3	4	5	6	7
C	2	3	5	7	9	9	11

$$C[A[8]] = C[A[8]] - 1$$

C	2	3	5	7	9	9	10
---	---	---	---	---	---	---	----

Counting Sort

	1	2	3	4	5	6	7	8	9	10	11
Input A[1..n]	7	1	3	1	2	4	5	7	2	4	3

	1	2	3	4	5	6	7	8	9	10	11
Output B[1..n]				2		3		4	5		7

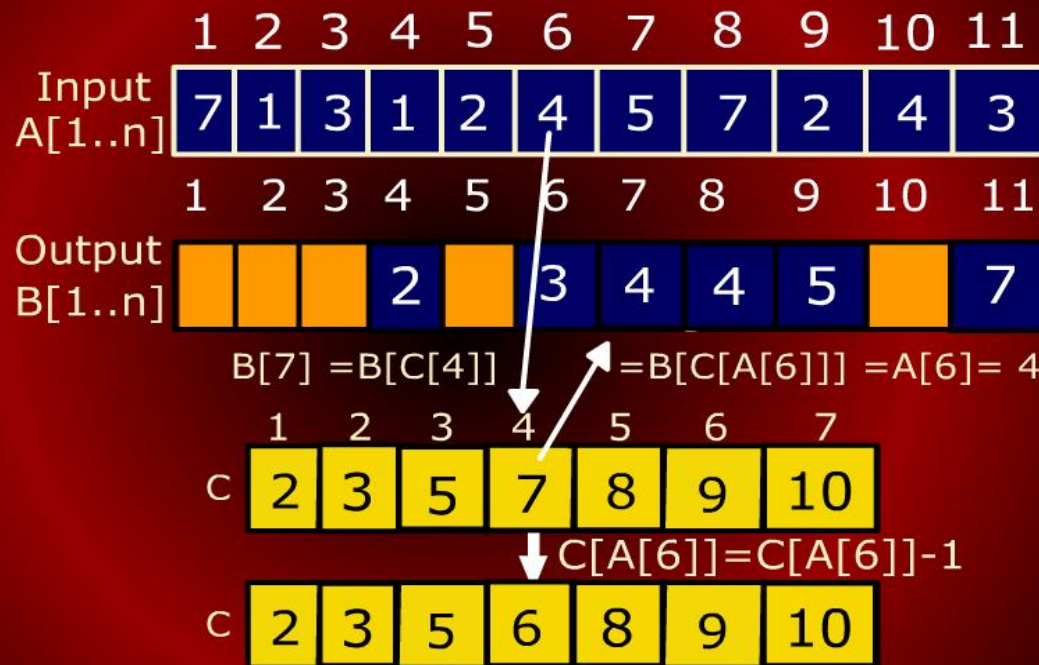
$$B[9] = B[C[5]] = B[C[A[7]]] = A[7] = 5$$

	1	2	3	4	5	6	7
C	2	3	5	7	9	9	10

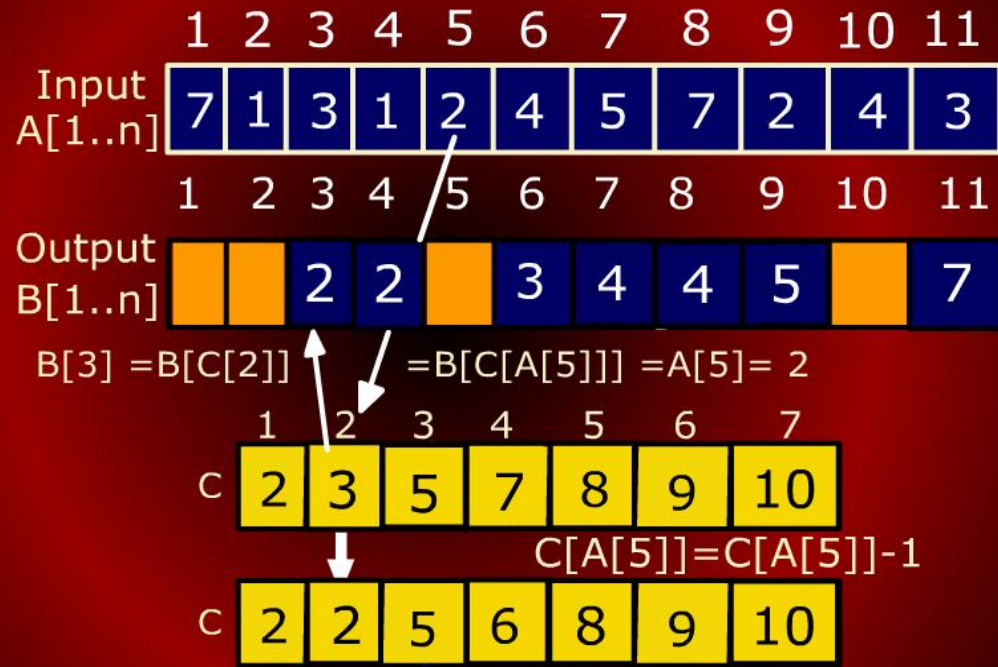
$$C[A[5]] = C[A[5]] - 1$$

C	2	3	5	7	8	9	10
---	---	---	---	---	---	---	----

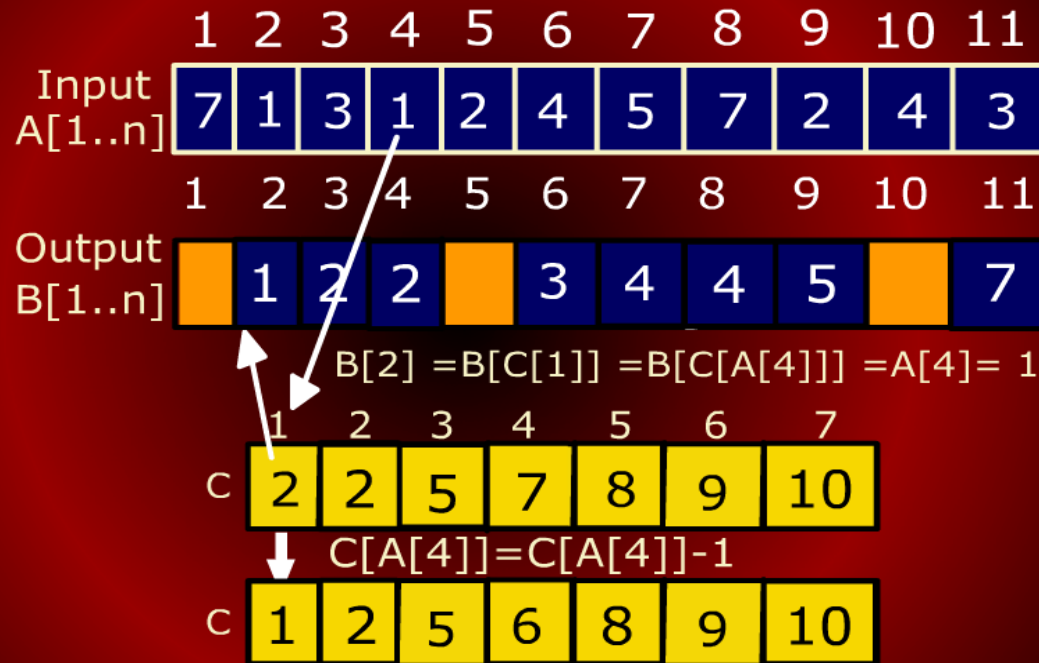
Counting Sort



Counting Sort



Counting Sort



Counting Sort

	1	2	3	4	5	6	7	8	9	10	11
Input A[1..n]	7	1	3	1	2	4	5	7	2	4	3

	1	2	3	4	5	6	7	8	9	10	11
Output B[1..n]		1	2	2	3	3	4	4	5		7

$$B[5] = B[C[3]] = B[C[A[3]]] = A[3] = 3$$

	1	2	3	4	5	6	7
C	1	2	5	7	8	9	10

$$C[A[3]] = C[A[3]] - 1$$

C	0	2	4	6	8	9	10
---	---	---	---	---	---	---	----

Counting Sort

	1	2	3	4	5	6	7	8	9	10	11
Input A[1..n]	7	1	3	1	2	4	5	7	2	4	3

	1	2	3	4	5	6	7	8	9	10	11
Output B[1..n]	1	1	2	2	3	3	4	4	5		7

$B[5] = B[1] = B[C[1]] = B[C[A[2]]] = A[2] = 1$

	1	2	3	4	5	6	7
C	1	2	4	7	8	9	10

$C[A[3]] = C[A[3]] - 1$

C	0	2	4	6	8	9	10
---	---	---	---	---	---	---	----

Counting Sort

	1	2	3	4	5	6	7	8	9	10	11
Input A[1..n]	7	1	3	1	2	4	5	7	2	4	3

	1	2	3	4	5	6	7	8	9	10	11
Output B[1..n]	1	1	2	2	3	3	4	4	5	7	7

$B[10] = B[C[7]] = B[C[A[1]]] = A[1] = 7$

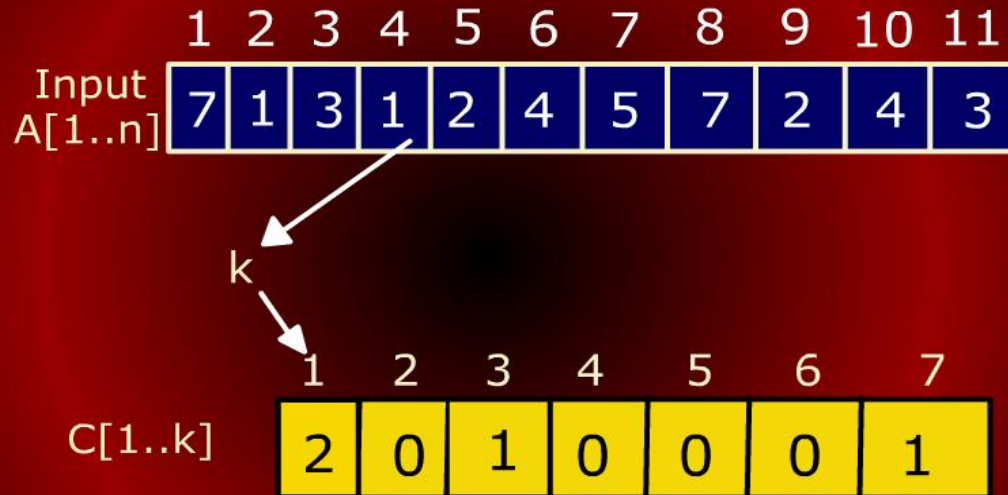
	1	2	3	4	5	6	7
C	0	2	4	7	8	9	10

$C[A[1]] = C[A[1]] - 1$

C	0	2	4	6	8	9	9
---	---	---	---	---	---	---	---

Counting sort

Counting Sort



Counting Sort

```
5  // C[i] now contains the number  
   of elements = i  
6  for i ← 2 to k  
7  do C[i] ← C[i] + C[i - 1]  k times  
8  // C[i] now contains the number  
   of elements ≤ i  
9  for j ← length[A] downto 1  
10 do B[C[A[j]]] A[j]  
11 C[A[j]] C[A[j]] - 1  n times
```

In-place, Stable Sorting

In-place, Stable Sorting

- Is it possible to do better than $O(n \log n)$?
- If a sorting algorithm is solely based on comparison of keys in the array then it is *impossible* to sort more efficiently than $\Omega(n \log n)$ time.
- All algorithms we have seen so far are comparison-based sorting algorithms.

In-place, Stable Sorting

In-place, Stable Sorting

algorithms but selection sort cannot (without significant modifications).

- Mergesort is a stable algorithm but not an in-place algorithm. It requires extra array storage.
- Quicksort is not stable but is an in-place algorithm.
- Heapsort is an in-place algorithm but is not stable.

Lower Bound for Sorting

Stable sorting



Design and Analysis of Algorithms